Big M Retail Database: SQL Implementation

Data Management | Team project (3 students) | MySQL and phpMyAdmin

Overview. Big M is a retail chain with stores, departments, employees, customers, products and orders. Starting from a relational design, we built the database in MySQL, loaded the supplied data, wrote ten analytical queries, and ran four data manipulation statements. This document summarises the design, the code, and the results.

1. Database design and creation

We created the database `BigM_157` with 10 tables and defined every primary key, foreign key and data type in SQL. The physical model below was generated from the finished database.

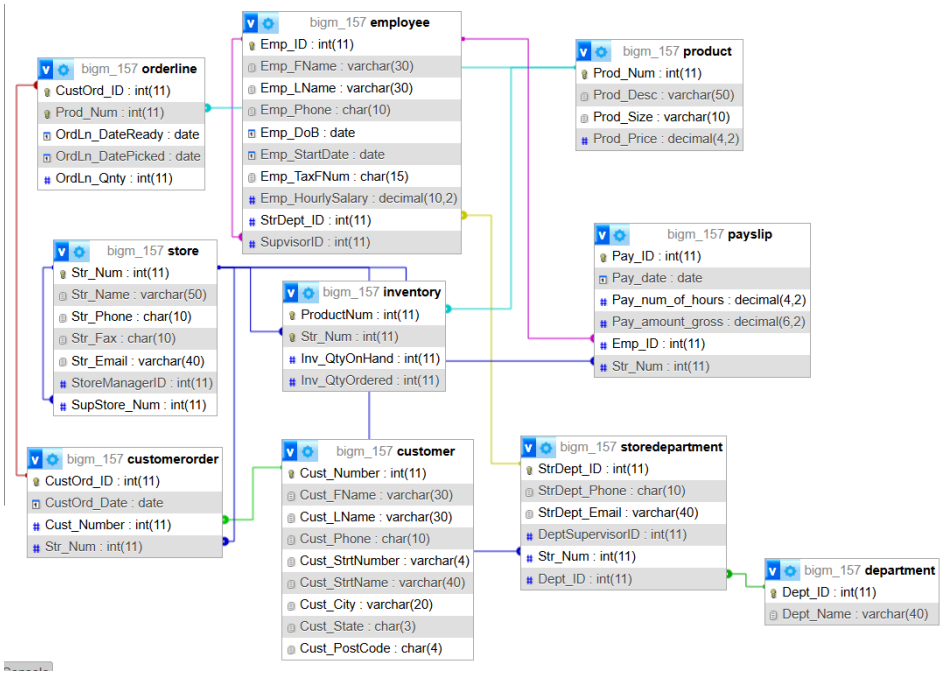


Figure 1: Physical model (designer view) of the Big M database

Table	Primary key	Foreign keys
CUSTOMER	Cust_Number	None
DEPARTMENT	Dept_ID	None
PRODUCT	Prod_Num	None
STORE	Str_Num	SupStore_Num to STORE (self-reference)
STOREDEPARTMENT	StrDept_ID	Str_Num to STORE, Dept_ID to DEPARTMENT
EMPLOYEE	Emp_ID	StrDept_ID to STOREDEPARTMENT, SupvisorID to EMPLOYEE (self-reference)
PAYSLIP	Pay_ID	Emp_ID to EMPLOYEE, Str_Num to STORE
INVENTORY	(ProductNum, Str_Num)	ProductNum to PRODUCT, Str_Num to STORE
CUSTOMERORDER	CustOrd_ID	Cust_Number to CUSTOMER, Str_Num to STORE
ORDERLINE	(CustOrd_ID, Prod_Num)	CustOrd_ID to CUSTOMERORDER, Prod_Num to PRODUCT

Design decisions

- **Many-to-many relationships resolved with junction tables.** INVENTORY links stores and products (stock per store), and ORDERLINE links orders and products. Both use composite primary keys.
- **Self-referencing foreign keys** model hierarchies: a store can report to a parent store, and an employee to a supervisor.
- **Data types chosen for the data:** `CHAR` for fixed-length values (phone, postcode, state), `VARCHAR` for names, `DECIMAL` for money and hours, `DATE` for dates, and `AUTO_INCREMENT` surrogate keys.
- Store manager and department supervisor IDs are plain integer columns without a foreign key constraint, so stores and departments can be loaded before employees.

Example DDL (self-reference, composite key)

```
CREATE TABLE IF NOT EXISTS EMPLOYEE (
    Emp_ID           INT PRIMARY KEY AUTO_INCREMENT,
    Emp_FName        VARCHAR(30),
    Emp_LName        VARCHAR(30),
    Emp_HourlySalary DECIMAL(10,2),
    StrDept_ID       INT,
    SupvisorID       INT,
    FOREIGN KEY (StrDept_ID) REFERENCES STOREDEPARTMENT(StrDept_ID),
    FOREIGN KEY (SupvisorID) REFERENCES EMPLOYEE(Emp_ID)
);

CREATE TABLE IF NOT EXISTS ORDERLINE (
    CustOrd_ID       INT,
    Prod_Num         INT,
    OrdLn_DateReady  DATE,
    OrdLn_DatePicked DATE,
    OrdLn_Qnty       INT,
    PRIMARY KEY (CustOrd_ID, Prod_Num),
    FOREIGN KEY (CustOrd_ID) REFERENCES CUSTOMERORDER(CustOrd_ID),
    FOREIGN KEY (Prod_Num)   REFERENCES PRODUCT(Prod_Num)
);
```

2. Loading the data

The supplied data was inserted with `INSERT ... VALUES` scripts, one per table. Tables were loaded in dependency order (parent tables before child tables) so that no foreign key was violated. Within STORE, the parent stores (1 to 3) were inserted before the stores that report to them.

Order	Table	Rows loaded
1	CUSTOMER, DEPARTMENT, PRODUCT (no dependencies)	10, 12, 12
2	STORE	10
3	STOREDEPARTMENT	48
4	EMPLOYEE	10
5	PAYSLIP, INVENTORY	9, 12
6	CUSTOMERORDER, ORDERLINE	9, 9

Each table was checked afterwards by viewing its contents, to confirm the row counts and values.

3. Analytical queries

Ten queries answer business questions using joins, subqueries, aggregation and formatting functions. Each is shown with its purpose, code and output.

Query I

List each employee's full name and hourly salary, lowest first.

```
SELECT CONCAT(Emp_FName, ' ', Emp_LName) AS 'Full Name',
       CONCAT('$ ', Emp_HourlySalary) AS 'Hourly Salary'
FROM EMPLOYEE
ORDER BY Emp_HourlySalary ASC;
```

Full Name	Hourly Salary
David Ross	$ 19.99
Asish Nehera	$ 29.99
Mohammad Ali	$ 29.99
Mohammad Ali	$ 39.99
Susan McClish	$ 39.99
Ahmed Ali	$ 49.99
David Ross	$ 79.99
Clive Simmonds	$ 89.99
Ahmed Ali	$ 109.99
David Sam	$ 209.99

Query II

Find the customer who placed the most recent order, with the date formatted day/month/year.

```
SELECT CONCAT(customer.Cust_FName, ' ', customer.Cust_LName) AS customer_name,
       DATE_FORMAT(CustOrd_Date, '%d/%m/%Y') AS Customer_Order_Date
FROM customerorder
JOIN customer ON customerorder.Cust_Number = customer.Cust_Number
ORDER BY customerorder.CustOrd_Date DESC
LIMIT 1;
```

customer_name	Customer_Order_Date
John Fraser	01/02/2026

Query III

Show each store with its manager's ID and full name, ordered by store name.

```
SELECT S.Str_Name, S.StoreManagerID,
       CONCAT(E.Emp_Fname, ' ', E.Emp_Lname) AS Emp_FullName
FROM store AS S, employee AS E
WHERE S.StoreManagerID = E.Emp_ID
ORDER BY Str_Name ASC;
```

	Str_Name	StoreManagerID	Emp_FullName
Edit Copy Delete	Ballarat	1	Mohammad Ali
Edit Copy Delete	Melbourne	9	Susan McClish
Edit Copy Delete	Point Cook	8	Asish Nehera
Edit Copy Delete	Sydney	10	David Sam
Edit Copy Delete	Werribee	7	Clive Simmonds

Query IV

Find customers who have never placed an order (subquery with NOT IN).

```
SELECT Cust_Number AS 'Customer Number',
       CONCAT(Cust_FName, ' ', Cust_LName) AS 'Full Name'
FROM CUSTOMER
WHERE Cust_Number NOT IN (
    SELECT DISTINCT Cust_Number FROM CUSTOMERORDER);
```

	Customer Number	Full Name
Edit Copy Delete	8	Tom Moody
Edit Copy Delete	9	Sobuj Barua
Edit Copy Delete	10	Carol Nimo

Query V

List the supervisor (an employee with no supervisor of their own) of the Sports department in each store.

```
SELECT store.Str_Name,
       CONCAT(employee.Emp_FName, ' ', employee.Emp_LName) AS SupervisorName
FROM employee
JOIN storedepartment ON employee.StrDept_ID = storedepartment.StrDept_ID
JOIN store ON storedepartment.Str_Num = store.Str_Num
JOIN department ON storedepartment.Dept_ID = department.Dept_ID
WHERE department.Dept_Name = 'Sports'
  AND employee.SupvisorID IS NULL;
```

Str_Name	SupervisorName
Ballarat	Mohammad Ali
Hobart	Mohammad Ali
Point Cook	Clive Simmonds
Darwin	David Sam

Query VI

Calculate the available quantity of each product across all stores (quantity on hand minus quantity ordered).

```
SELECT p.Prod_Num, p.Prod_Desc,
       SUM(i.Inv_QtyOnHand) - SUM(i.Inv_QtyOrdered) AS AvailableQuantity
FROM PRODUCT AS p
JOIN INVENTORY AS i ON p.Prod_Num = i.ProductNum
GROUP BY p.Prod_Num, p.Prod_Desc;
```

Prod_Num	Prod_Desc	AvailableQuantity
1	Shirt	55
2	Shirt	20
3	Pant	49
4	Pant	38
5	T-shirt	53
6	Skirt	50
7	Shirt	101
8	Shirt	51

Query VII

Total quantity sold per product, counting only order lines picked up on or before 20 May 2026.

```
SELECT P.Prod_Num AS 'Product Number', P.Prod_Desc AS 'Description',
       SUM(OL.OrdLn_Qnty) AS 'Total Quantity Sold'
FROM PRODUCT AS P
JOIN ORDERLINE AS OL ON P.Prod_Num = OL.Prod_Num
WHERE OL.OrdLn_DatePicked <= '2026-05-20'
GROUP BY P.Prod_Num, P.Prod_Desc
ORDER BY P.Prod_Num, SUM(OL.OrdLn_Qnty);
```

Product Number	Description	Total Quantity Sold
1	Shirt	5
2	Shirt	3
3	Pant	15
4	Pant	20
5	T-shirt	1
6	Skirt	2
7	Shirt	5
8	Shirt	5
10	Pant	5

Query VIII

Find products priced at or below the average product price (subquery with AVG).

```
SELECT Prod_Num, Prod_Desc,
       CONCAT('$', CAST(Prod_Price AS DECIMAL(7,2))) AS Prod_Price
FROM product
WHERE Prod_Price <= (SELECT AVG(Prod_Price) FROM product);
```

	Prod_Num	Prod_Desc	Prod_Price
☐ Edit Copy Delete	1	Shirt	$25.99
☐ Edit Copy Delete	3	Pant	$25.99
☐ Edit Copy Delete	5	T-shirt	$15.99
☐ Edit Copy Delete	7	Shirt	$25.99
☐ Edit Copy Delete	9	Pant	$25.99
☐ Edit Copy Delete	11	T-shirt	$15.99

Query IX

Show every employee's new hourly salary after a 7.5% increase, ordered by name.

```
SELECT CONCAT(E.Emp_Fname, ' ', E.Emp_Lname) AS Full_Name,
       CONCAT('$ ', CAST(E.Emp_HourlySalary * 1.075 AS DECIMAL(10,2))) AS New_Salary
FROM employee AS E
ORDER BY Full_Name;
```

Full_Name ▲ 1	New_Salary
Ahmed Ali	$ 53.74
Ahmed Ali	$ 118.24
Asish Nehera	$ 32.24
Clive Simmonds	$ 96.74
David Ross	$ 21.49
David Ross	$ 85.99
David Sam	$ 225.74
Mohammad Ali	$ 42.99
Mohammad Ali	$ 32.24
Susan McClish	$ 42.99

Query X

Show hours and gross pay for the most recent pay date (subquery with MAX).

```
SELECT CONCAT(E.Emp_FName, ' ', E.Emp_LName) AS 'Full Name',
       CONCAT('$', P.Pay_num_of_hours) AS 'Hours Paid',
       CONCAT('$', P.Pay_amount_gross) AS 'Amount Paid'
FROM PAYSLIP P
JOIN EMPLOYEE AS E ON P.Emp_ID = E.Emp_ID
WHERE P.Pay_date = (SELECT MAX(Pay_date) FROM PAYSLIP);
```

Full Name	Hours Paid	Amount Paid
Clive Simmonds	$30.00	$450.50
Asish Nehera	$32.00	$355.50
Susan McClish	$28.00	$255.50

4. Data manipulation statements

Four statements simulate a new customer placing an order that is later collected. The order of the statements follows the foreign key chain: customer, then order, then order line. The affected rows are highlighted in each output.

Statement I

Insert a new customer. The database auto-assigned Cust_Number 11.

```
INSERT INTO customer (Cust_FName, Cust_LName, Cust_Phone, Cust_StrtNumber,
    Cust_StrtName, Cust_City, Cust_State, Cust_PostCode)
VALUES ('Daniel', 'Ortega', '0431245668', '30', 'Taylor Place',
    'Brisbane', 'Queensland', '4156');
```

	Cust_Number	Cust_FName	Cust_LName	Cust_Phone	Cust_StrtNumber	Cust_StrtName	Cust_City	Cust_State	Cust_PostCode
Edit Copy Delete	1	John	Fraser	0345567675	10	Technology Lane	Nathan	QLD	4111
Edit Copy Delete	2	Mohammad	Awrangjeb	0351226462	15	Westmeadows Lane	Sunnybank	QLD	4111
Edit Copy Delete	3	Ali	Abdullah	0351457462	22	Adam Street	Woodridge	NSW	2006
Edit Copy Delete	4	Jessica	Fayaj	0255665462	15	Rose Drive	Robertson	NSW	2122
Edit Copy Delete	5	Karim	Hamaoud	0351226444	16	Winifred Crescent	Southport	VIC	3029
Edit Copy Delete	6	Partho	Pratim	0351226666	11	Station Road	Arundell	VIC	3842
Edit Copy Delete	7	Clive	Perth	0789526462	30	Wisdom Place	Inala	WA	6505
Edit Copy Delete	8	Tom	Moody	0151247895	52	Technology Highway	Durrack	WA	6666
Edit Copy Delete	9	Sobuj	Barua	0387626462	84	West Ring Road	Karawatha	SA	5555
Edit Copy Delete	10	Carol	Nimo	0345545673	17	East Corner Lane	Calamvale	SA	5000
Edit Copy Delete	13	Daniel	Ortega	0431245668	30	Taylor Place	Brisbane	Que	4156

Statement II

Insert an order for the new customer (11) at store 7. The order was auto-numbered 10.

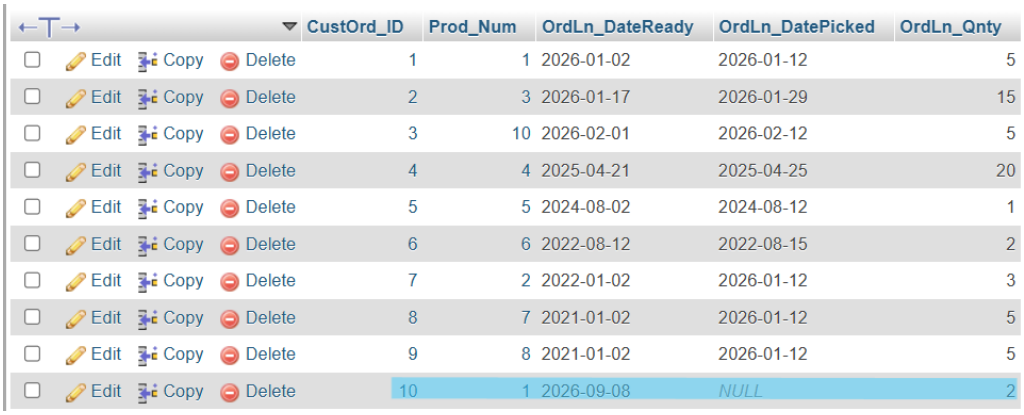
```
INSERT INTO customerorder (CustOrd_Date, Cust_Number, Str_Num)
VALUES ('2026-09-06', 11, 7);
```

	CustOrd_ID	CustOrd_Date	Cust_Number	Str_Num
Edit Copy Delete	1	2026-01-01	1	1
Edit Copy Delete	2	2026-01-15	1	3
Edit Copy Delete	3	2026-02-01	1	10
Edit Copy Delete	4	2025-04-21	2	4
Edit Copy Delete	5	2024-07-31	3	5
Edit Copy Delete	6	2022-08-11	4	6
Edit Copy Delete	7	2022-06-10	5	2
Edit Copy Delete	8	2021-05-12	6	7
Edit Copy Delete	9	2021-10-24	7	8
Edit Copy Delete	10	2026-09-06	11	7

Statement III

Add an order line for 2 units of product 1, ready on 8 September and not yet picked up (NULL).

```
INSERT INTO orderline (CustOrd_ID, Prod_Num, OrdLn_DateReady,
    OrdLn_DatePicked, OrdLn_Qnty)
VALUES (10, 1, '2026-09-08', NULL, 2);
```

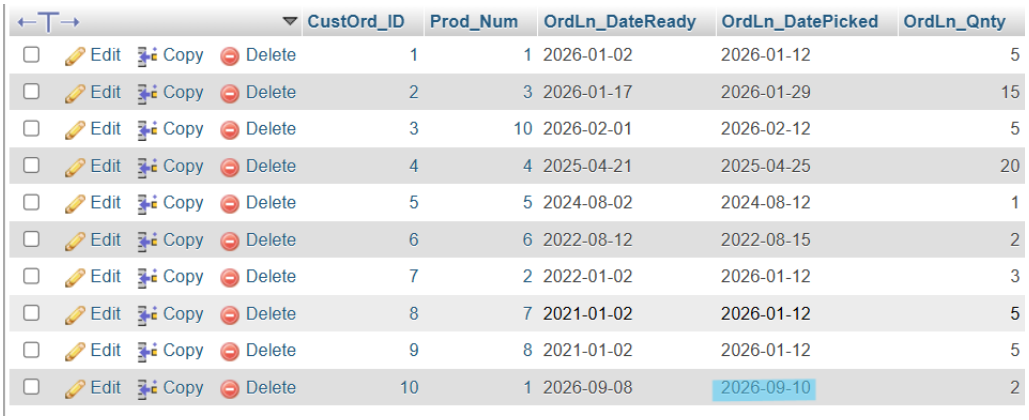


	CustOrd_ID	Prod_Num	OrdLn_DateReady	OrdLn_DatePicked	OrdLn_Qnty
Edit Copy Delete	1	1	2026-01-02	2026-01-12	5
Edit Copy Delete	2	3	2026-01-17	2026-01-29	15
Edit Copy Delete	3	10	2026-02-01	2026-02-12	5
Edit Copy Delete	4	4	2025-04-21	2025-04-25	20
Edit Copy Delete	5	5	2024-08-02	2024-08-12	1
Edit Copy Delete	6	6	2022-08-12	2022-08-15	2
Edit Copy Delete	7	2	2022-01-02	2026-01-12	3
Edit Copy Delete	8	7	2021-01-02	2026-01-12	5
Edit Copy Delete	9	8	2021-01-02	2026-01-12	5
Edit Copy Delete	10	1	2026-09-08	NULL	2

Statement IV

Record the pick-up date once the customer collects the order.

```
UPDATE orderline
SET OrdLn_DatePicked = '2026-09-10'
WHERE CustOrd_ID = 10 AND Prod_Num = 1;
```



	CustOrd_ID	Prod_Num	OrdLn_DateReady	OrdLn_DatePicked	OrdLn_Qnty
Edit Copy Delete	1	1	2026-01-02	2026-01-12	5
Edit Copy Delete	2	3	2026-01-17	2026-01-29	15
Edit Copy Delete	3	10	2026-02-01	2026-02-12	5
Edit Copy Delete	4	4	2025-04-21	2025-04-25	20
Edit Copy Delete	5	5	2024-08-02	2024-08-12	1
Edit Copy Delete	6	6	2022-08-12	2022-08-15	2
Edit Copy Delete	7	2	2022-01-02	2026-01-12	3
Edit Copy Delete	8	7	2021-01-02	2026-01-12	5
Edit Copy Delete	9	8	2021-01-02	2026-01-12	5
Edit Copy Delete	10	1	2026-09-08	2026-09-10	2

Note: the state was entered as "Queensland", but the `Cust_State` column is `CHAR(3)`, so MySQL stored only "Que". A three-letter code such as "QLD" would store correctly, which matches the rest of the customer data.

Skills demonstrated

Relational database design, ER modelling, SQL DDL (tables, primary and foreign keys, composite keys, self-references), SQL DML (INSERT, UPDATE), multi-table joins, subqueries, aggregation with GROUP BY, string, date and numeric formatting functions, MySQL and phpMyAdmin.